SmartDistServe: Reproducible Simulation of Deadline-Aware Scheduling in Disaggregated LLM Serving

Research manuscript | 13 October 2026

Abstract

Disaggregated large language model serving separates prompt processing from autoregressive decoding, but request heterogeneity continues to influence queueing delay and service-level objective (SLO) attainment. This study presents SmartDistServe, a reproducible discrete-event simulation prototype comparing phase-local first-come-first-served (FCFS), estimated shortest-job-first (SJF), earliest-deadline-first (EDF), and a heuristic combining normalized deadline slack, estimated request duration, and aging. Evaluation comprises 440 policy runs on 110 stored synthetic trace instances using five seeds. Metrics include time to first token, time per output token, joint SLO attainment, cohort goodput per GPU, tail waiting time, and group-outcome fairness. Across 16 main conditions, the heuristic exceeds FCFS in mean attainment in four conditions, falls below it in four, and ties in eight. Under mixed overload at six requests per second, estimated SJF achieves 55.30% mean attainment compared with 13.45% for the heuristic, while its mean run-level P95 waiting time increases from 28.10 to 61.71 seconds. The results identify performance tradeoffs rather than universal policy superiority. All workloads are synthetic and service coefficients are uncalibrated; improvement over physical serving systems is not established.

Keywords: LLM serving; disaggregation; discrete-event simulation; deadline scheduling; SLO attainment; fairness.

1. Introduction

DistServe (Zhong et al., OSDI 2024) [1] separates prefill and decoding onto different GPUs, uses FCFS runtime ordering with queue/load-based routing, and optimizes placement and resource allocation. Its runtime section discusses convoy effects and complementary scheduling changes. SmartDistServe adopts separate phase resources, first-token generation during prefill, and a simplified shortest-queue/least-occupancy FCFS reference. It does not implement DistServe's placement optimizer, tensor/pipeline parallelism, CUDA kernels, KV memory manager, or engine. Comparing policies here does not measure improvement over the published DistServe system.

2. Related Work and Contribution

Priority, short-job scheduling, deadline ordering, and aging are established techniques. Prior LLM-serving studies address output-length prediction, request prioritization, and differentiated service objectives:

- Qiu et al., Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction (2024) [2] uses a proxy predictor for speculative SJF.
- Fu et al., Efficient LLM Scheduling by Learning to Rank (2024) [3] predicts relative output-length ranks for scheduling.
- Sun et al., Llumnix (OSDI 2024) [4] supports runtime rescheduling, differentiated priorities, and SLOs across instances.
- Huang et al., SLO-Aware Scheduling for Large Language Model Inferences (2025) [5] considers request SLOs, input lengths, and possible output lengths when choosing priority order.
- Wu et al., FastServe (NSDI 2026) [6] uses token-level preemption and a skip-join feedback scheduler. This prototype does not reproduce its preemption or offload machinery.

The contribution is a reproducible experimental artifact combining these ideas with explicit disaggregated queues and fairness diagnostics. Its particular score is a study heuristic, with no proof of optimality or literature-backed

novelty claim. Experiments compare only FCFS, estimated SJF, and EDF, not implementations of the related systems above. Establishing a stronger contribution would require calibrated inference measurements, actual traces, stronger scheduling baselines, and a broader literature review.

3. System Model and Scheduling Policies

Two prefill GPUs and one decode GPU are provisioned in the reference configuration. A request routes at arrival to the prefill GPU with the smallest number of queued plus running requests; ties use GPU ID. Each GPU processes one prefill at a time, nonpreemptively. Prefill completion emits token 1. The modeled KV transfer then completes independently, with no shared-link contention. A request routes to the decode GPU with the fewest active plus queued requests. Decode admits requests between iterations into at most 16 active slots and generates one token per active request per iteration. An admitted request keeps its slot until completion; priority only changes admission. At coincident timestamps, completions release resources before transfers and arrivals route, and all events are processed before dispatch. All policies share routing, batching, hardware, SLOs, and realized traces. They do not drop requests or cancel requests with expired deadlines.

For prompt length L, current batch B, and maximum active context C:

```
prefill(L) = 0.005 + L/8000 + 0.01*(L/4096)^2          seconds
transfer(L) = 0.002 + L*131072/25000000000             seconds
decode_step(B,C) = 0.012 + 0.0006*(B-1) + 0.00000025*C seconds
```

These functions are deliberately simple synthetic cost assumptions. GPU counts mean one abstract worker per GPU; model parallelism and model-fit feasibility are not represented.

FCFS orders by phase enqueue time. Estimated SJF orders by estimated service for the phase (prefill duration or predicted remaining decode duration). EDF orders by the phase deadline. Prefill deadline is arrival plus TTFT SLO. Decode uses a proxy completion deadline `first_token + max(1, estimated_output_tokens-1)*TPOT_SLO`; exact output length is unknown to the scheduler, so this is not an oracle EDF or a guarantee of TPOT success.

Let e be phase enqueue time, d the phase deadline, s the estimated remaining phase service, D the estimated duration (prefill + transfer + decode before prefill, remaining decode before decode admission), and b the phase SLO budget. At time t, Smart selects the smallest score:

```
slack = d - t - s
score = (slack + 0.1*D - 0.2*(t-e)) / b
```

Each score normalizes by its request's budget: TTFT SLO for prefill, estimated post-first-token count times TPOT SLO for decode. Negative slack remains eligible and can get high priority. Before scoring, any request waiting at least five seconds is promoted; the oldest eligible request wins. Ties use enqueue time then request ID. This aging rule gives eventual admission with bounded service and a finite number of older requests, but it provides no fixed waiting-time bound under overload and cannot interrupt long running prefill or decode requests. SJF and EDF do not receive aging; the ablation removes it from Smart to expose that difference.

4. Workloads and Duration Estimation

The complete synthetic distribution configuration is summarized below. Bounds are inclusive; sigma denotes the lognormal shape parameter.

Profile	Input mean	Output mean	Input bounds	Output bounds	Input sigma	Output sigma	TTFT / TPOT s
ShareGPT	755	200	16-8192	2-1024	0.9	0.8	1.5 / 0.08
HumanEval	171	98	16-2048	2-512	0.7	0.7	0.35 / 0.10
LongBench	8192	91	1024-32768	2-512	0.6	0.6	8.0 / 0.06

ShareGPT-inspired chat, HumanEval-inspired code, and LongBench-inspired long-context profiles are **generated distributions**, not samples from these datasets. No data download, tokenizer, or benchmark evaluation occurred.

Input/output lengths are independent rounded, clipped lognormal samples; configured means are pre-clipping means. Default `(input mean, output mean, TTFT SLO seconds, TPOT SLO seconds)` are `(755,200,1.5,0.08)`, `(171,98,0.35,0.10)`, and `(8192,91,8.0,0.06)`. The long profile is intentionally a long-context scenario, not DistServe's context-capped LongBench workload. Distribution widths, bounds, and SLOs are configurable in `configs/experiment.json`.

Prefill estimates use known input length and the cost model, multiplied by mean-one lognormal noise with sigma 0.3. Output estimates use only the profile mean and independently sampled lognormal noise, not realized output tokens. Therefore sigma zero gives exact modeled prefill and a profile output prior, not perfect output-length prediction. No learned predictor is implemented or claimed. Separate arrival, length, and estimate random streams preserve realized arrivals and lengths across predictor-noise ablations.

Arrivals follow a Poisson process unless the burst ablation enables a periodic two-level rate via thinning. Its low/high rates are normalized to the configured average over a full period. The main grid uses four rates per profile and mixed workload. Mixed request probabilities are 50% chat, 25% code, and 25% long-context. Each run accepts arrivals for 90 simulated seconds. Only arrivals in `[15,90)` enter the measured cohort; the first 15 seconds warm up queues, and every arrival is fully drained afterward. The measurement is transient and finite, especially at overloaded rates. Five seeds are shared across all policies. Ablations at mixed rate 3 req/s remove aging, remove the duration term, use a profile prior without extra noise, increase noise, enable bursts, and slow KV transfer. No parameters were selected to force a desired outcome.

5. Evaluation Metrics

- TTFT = first-token timestamp minus arrival, including prefill queue wait.
- TPOT = `(completion-first_token)/(output_tokens-1)`, including transfer and decode admission wait. One-token requests have undefined TPOT, omitted from TPOT averages and treated as satisfying the TPOT constraint. Max observed token gap is retained because average TPOT can hide a long initial decode stall.
- Joint SLO attainment = fraction of cohort requests satisfying both TTFT and average TPOT limits, with a `1e-12`-second numerical boundary tolerance.
- Cohort per-GPU goodput = joint successes divided by `(90-15)` seconds and all provisioned GPUs. It counts successes for cohort arrivals even if they finish during drain. It is neither token throughput nor steady-state maximum sustainable rate.
- Waiting = prefill queue wait plus decode admission wait. Transfer and active service are excluded; P95 uses linear interpolation.
- Fairness = Jain index over per-profile and fixed prompt-size-bin joint SLO attainment rates. Prompt bins are `<=512`, `513..2048`, and `>2048`; empty bins are omitted. All-zero group attainment maps to zero. Single populated-group Jain equals one and cannot establish fairness; differing per-group SLOs also limit interpretation. Per-group counts, waiting tails, max wait, and P95 size-normalized slowdown supplement the index.
- Slowdown uses end-to-end latency divided by this model's isolated request service; it is a diagnostic, not a tenant service-share guarantee.

6. Experimental Evaluation

The evaluated workload distributions and service coefficients are synthetic assumptions. No physical inference engine or empirical benchmark dataset was used. All reported numerical outcomes originate from the executed simulator and saved experiment artifacts.

Executed 440 policy runs on 110 synthetic traces. Seeds: [11, 22, 33, 44, 55]. All results below were computed from `runs.json`.

Each cell is a mean across seeds. Intervals in `summary.csv` are two-sided 95% Student t intervals on seed-level statistics; tail percentiles are averaged across runs, not pooled. These intervals describe workload sampling variability, not hardware-model uncertainty. No multiple-comparison correction is applied.

6.1. Complete Main-Sweep Results

Synthetic profile	Rate req/s	Policy	Mean TTFT s	Mean TPOT s	SLO %	Goodput req/s/GPU	P95 wait s	Jain prompt-bin SLO
humaneval	2	edf	0.026	0.014	100.0	0.685	0.013	1.000
humaneval	2	fcfs	0.026	0.014	100.0	0.685	0.013	1.000
humaneval	2	sjf	0.026	0.014	100.0	0.685	0.013	1.000
humaneval	2	smart	0.026	0.014	100.0	0.685	0.013	1.000
humaneval	4	edf	0.026	0.016	100.0	1.367	0.015	1.000
humaneval	4	fcfs	0.026	0.016	100.0	1.367	0.015	1.000
humaneval	4	sjf	0.026	0.016	100.0	1.367	0.015	1.000
humaneval	4	smart	0.026	0.016	100.0	1.367	0.015	1.000
humaneval	8	edf	0.027	0.088	70.4	1.886	8.969	0.998
humaneval	8	fcfs	0.027	0.088	70.7	1.894	6.290	0.997
humaneval	8	sjf	0.027	0.081	89.0	2.408	28.416	0.999
humaneval	8	smart	0.027	0.088	70.5	1.887	6.360	0.995
humaneval	16	edf	0.028	1.005	0.9	0.047	93.112	0.578
humaneval	16	fcfs	0.028	1.003	0.7	0.036	92.007	0.554
humaneval	16	sjf	0.028	0.842	43.3	2.336	133.102	0.997
humaneval	16	smart	0.028	1.003	0.7	0.036	92.008	0.554
longbench	0.5	edf	1.086	0.015	100.0	0.166	0.265	1.000
longbench	0.5	fcfs	1.086	0.015	100.0	0.166	0.265	1.000
longbench	0.5	sjf	1.086	0.015	100.0	0.166	0.265	1.000
longbench	0.5	smart	1.086	0.015	100.0	0.166	0.265	1.000
longbench	1	edf	1.173	0.016	100.0	0.315	1.189	1.000
longbench	1	fcfs	1.173	0.016	100.0	0.315	1.189	1.000
longbench	1	sjf	1.171	0.016	100.0	0.315	1.208	1.000
longbench	1	smart	1.176	0.016	100.0	0.315	1.183	1.000
longbench	2	edf	6.274	0.017	73.0	0.489	13.005	0.975
longbench	2	fcfs	6.274	0.017	73.0	0.489	13.005	0.975
longbench	2	sjf	3.981	0.017	90.4	0.618	15.168	0.997
longbench	2	smart	6.429	0.017	73.0	0.490	13.482	0.976
longbench	3	edf	32.503	0.018	2.3	0.023	55.579	0.353
longbench	3	fcfs	32.503	0.018	2.3	0.023	55.579	0.353
longbench	3	sjf	15.048	0.017	73.1	0.749	75.619	0.974
longbench	3	smart	32.583	0.018	0.9	0.009	56.112	0.200
mixed	1	edf	0.307	0.014	99.2	0.312	0.014	1.000
mixed	1	fcfs	0.307	0.014	99.2	0.312	0.014	1.000
mixed	1	sjf	0.307	0.014	99.2	0.312	0.014	1.000

Synthetic profile	Rate req/s	Policy	Mean TTFT s	Mean TPOT s	SLO %	Goodput req/s/GPU	P95 wait s	Jain prompt-bin SLO
mixed	1	smart	0.307	0.014	99.2	0.312	0.014	1.000
mixed	2	edf	0.442	0.017	96.4	0.660	0.810	0.999
mixed	2	fcfs	0.445	0.017	96.4	0.660	0.815	0.999
mixed	2	sjf	0.441	0.017	96.6	0.661	0.793	0.999
mixed	2	smart	0.445	0.017	96.3	0.660	0.806	0.999
mixed	3	edf	0.622	0.020	92.3	0.951	1.597	0.996
mixed	3	fcfs	0.679	0.020	89.6	0.922	1.840	0.994
mixed	3	sjf	0.612	0.021	92.5	0.953	1.693	0.997
mixed	3	smart	0.658	0.020	90.4	0.931	1.813	0.995
mixed	6	edf	2.855	0.178	25.6	0.520	30.222	0.923
mixed	6	fcfs	5.239	0.178	9.5	0.194	28.709	0.811
mixed	6	sjf	1.563	0.132	55.3	1.143	61.711	0.941
mixed	6	smart	4.706	0.179	13.4	0.273	28.096	0.871
sharegpt	1	edf	0.085	0.014	100.0	0.315	0.013	1.000
sharegpt	1	fcfs	0.085	0.014	100.0	0.315	0.013	1.000
sharegpt	1	sjf	0.085	0.014	100.0	0.315	0.013	1.000
sharegpt	1	smart	0.085	0.014	100.0	0.315	0.013	1.000
sharegpt	2	edf	0.100	0.017	100.0	0.685	0.017	1.000
sharegpt	2	fcfs	0.100	0.017	100.0	0.685	0.017	1.000
sharegpt	2	sjf	0.100	0.017	100.0	0.685	0.017	1.000
sharegpt	2	smart	0.100	0.017	100.0	0.685	0.017	1.000
sharegpt	4	edf	0.105	0.078	72.2	0.972	14.239	0.986
sharegpt	4	fcfs	0.105	0.077	71.7	0.964	10.692	0.992
sharegpt	4	sjf	0.105	0.080	84.0	1.141	33.609	0.996
sharegpt	4	smart	0.105	0.077	71.6	0.962	10.650	0.992
sharegpt	8	edf	0.126	0.598	4.3	0.116	100.186	0.831
sharegpt	8	fcfs	0.126	0.600	2.7	0.073	98.623	0.889
sharegpt	8	sjf	0.124	0.515	40.3	1.092	140.867	0.993
sharegpt	8	smart	0.127	0.600	2.7	0.073	98.616	0.883

6.2. Paired Comparisons with FCFS

Differences below are paired by seed and trace. Positive SLO difference favors Smart; positive waiting-time difference means Smart has worse tail waiting. These are descriptive comparisons over the entire main grid, not a selected best case.

Synthetic profile	Rate req/s	SLO difference, pp (95% halfwidth)	P95 wait difference, s (95% halfwidth)
humaneval	2	+0.000 +/- 0.000	+0.000 +/- 0.000
humaneval	4	+0.000 +/- 0.000	-0.000 +/- 0.000
humaneval	8	-0.268 +/- 0.933	+0.070 +/- 0.175
humaneval	16	+0.000 +/- 0.000	+0.001 +/- 0.014

Synthetic profile	Rate req/s	SLO difference, pp (95% halfwidth)	P95 wait difference, s (95% halfwidth)
longbench	0.5	+0.000 +/- 0.000	+0.000 +/- 0.000
longbench	1	+0.000 +/- 0.000	-0.006 +/- 0.017
longbench	2	+0.064 +/- 1.735	+0.477 +/- 0.499
longbench	3	-1.382 +/- 2.582	+0.533 +/- 2.753
mixed	1	+0.000 +/- 0.000	+0.000 +/- 0.000
mixed	2	-0.125 +/- 0.347	-0.009 +/- 0.020
mixed	3	+0.831 +/- 0.897	-0.026 +/- 0.191
mixed	6	+3.899 +/- 3.239	-0.613 +/- 0.989
sharegpt	1	+0.000 +/- 0.000	+0.000 +/- 0.000
sharegpt	2	+0.000 +/- 0.000	+0.000 +/- 0.000
sharegpt	4	-0.106 +/- 1.062	-0.042 +/- 0.116
sharegpt	8	+0.004 +/- 0.146	-0.008 +/- 0.047

Across 16 tested conditions, Smart's mean SLO attainment exceeds FCFS in 4, falls below it in 4, and ties in 8. Paired mean differences range from -1.382 to +3.899 percentage points. This does not establish universal superiority or novelty.

6.3. Performance Tradeoffs

At mixed rate 3 req/s, mean joint SLO attainment is 89.58% (FCFS), 90.41% (Smart), 92.50% (SJF), and 92.33% (EDF). Mean run P95 queue wait is 1.840 s (FCFS), 1.813 s (Smart), 1.693 s (SJF), and 1.597 s (EDF). Aggregate attainment and tail waiting therefore need to be assessed together; a higher successful-request rate alone does not establish better fairness.

At mixed rate 6 req/s, mean joint SLO attainment is 9.55% (FCFS), 13.45% (Smart), 55.30% (SJF), and 25.64% (EDF). Mean run P95 queue wait is 28.709 s (FCFS), 28.096 s (Smart), 61.711 s (SJF), and 30.222 s (EDF). Aggregate attainment and tail waiting therefore need to be assessed together; a higher successful-request rate alone does not establish better fairness.

Removing both aging mechanisms changes mean mixed-rate-3 Smart attainment from 90.41% to 90.14% (-0.264 percentage points). This is one operating point, so it does not establish a general effect or a statistically reliable component benefit.

Removing the duration term changes mean mixed-rate-3 Smart attainment from 90.41% to 91.52% (+1.111 percentage points). This is one operating point, so it does not establish a general effect or a statistically reliable component benefit.

6.4. Component and Sensitivity Ablations

Every ablation includes all four policies and identical traces within that condition. The table shows Smart only; full comparison data are retained in the summary and run files.

Condition	SLO %	P95 wait s	Jain profile SLO	Jain prompt-bin SLO
bursty_arrivals	87.3	2.470	0.980	0.992
no_aging	90.1	1.834	0.988	0.995
no_duration	91.5	1.709	0.989	0.996
noisy_estimates	90.6	1.789	0.988	0.996
profile_prior_only	90.7	1.781	0.987	0.995

Condition	SLO %	P95 wait s	Jain profile SLO	Jain prompt-bin SLO
slow_transfer	87.9	1.810	0.992	0.998

6.5. Tested-Grid Capacity Diagnostic

For each workload/policy, select the largest offered rate for which **every seed** attains at least 90% joint SLO success. Report offered rate divided by provisioned GPUs. This finite, fully drained simulation is not a steady-state capacity measurement, and this diagnostic is distinct from the cohort successful-request goodput above. 'None' means no tested rate qualified, not zero capacity.

Synthetic profile	Policy	Highest qualified tested rate req/s	Offered rate/GPU
humaneval	edf	4	1.333
humaneval	fcfs	4	1.333
humaneval	sjf	4	1.333
humaneval	smart	4	1.333
longbench	edf	1	0.333
longbench	fcfs	1	0.333
longbench	sjf	1	0.333
longbench	smart	1	0.333
mixed	edf	2	0.667
mixed	fcfs	2	0.667
mixed	sjf	2	0.667
mixed	smart	2	0.667
sharegpt	edf	2	0.667
sharegpt	fcfs	2	0.667
sharegpt	sjf	2	0.667
sharegpt	smart	2	0.667

The saved request logs include all warmup and measurement arrivals, failures, single-token cases, first-token timestamps, decode admission, and final completion. Use `python scripts/audit_results.py` to independently recompute the scalar metrics.

7. Limitations and Threats to Validity

The numerical results are conditional on the stated synthetic distributions, arbitrary cost model, SLOs, GPU allocation, batch limit, and heuristic weights. They do not estimate physical GPU throughput, DistServe speedup, dataset fidelity, energy, cost, or production viability. There is no KV capacity limit, memory fragmentation, swap, preemption, prefix reuse, speculative decoding, transfer contention, pipeline/tensor communication, admission rejection, model quality, tokenization, hardware variance, or scheduler computation overhead. Active decode and prefill work cannot be displaced. Shared local queues can cause routing imbalance; queue length is not remaining work. CPU runtime is not converted to simulated scheduling overhead.

The workload has no tenants beyond application groups, no correlated input/output lengths, and no learned estimates. Five seeds yield broad uncertainty for some cases. No confidence interval represents cost-model uncertainty. The suite is a fixed finite grid; capacity diagnostics do not locate the exact saturation boundary, and complete draining is not a substitute for long steady-state runs. Fairness indices of SLO outcomes differ from resource fairness and can rise even when all groups perform poorly. The reference reports absolute SLO rates and group details so this cannot be hidden by a single index.

Results establish only how the implemented policies behave within this simulator. Scheduling can change which requests meet their SLOs and can trade aggregate success against tail wait or group outcomes. The tables include regressions and ties. A claim that Smart always outperforms FCFS, SJF, or EDF is unsupported. The next research steps are measured service profiling, licensed trace acquisition with tokenizer provenance, learned or rank-based estimate evaluation, strong SLO/fair scheduling baselines, explicit memory and link models, load-duration sensitivity, and eventual GPU validation.

8. Reproducibility and Artifact Validation

The README supplies commands for tests, experiments, plots, independent audit, and exact metric replay. `manifest.json` records configuration, seeds, trace hashes, numeric output hashes, source hashes, Python version, and platform. `requests.csv.gz` contains per-request outcomes; `traces.jsonl.gz` contains complete input traces. `summary.csv` and `paired_contrasts.json` retain seed uncertainty. Graphs are generated only from measured JSON. `research/REVIEW.md` records the final code and result review, discovered issues, fixes, and checks actually executed. Passing these checks supports internal consistency, not real-world validity.

The final simulator passed 38 automated tests. Independent recomputation from serialized timelines completed 1,505,479 checks with 0 failures across 149,400 policy-specific request outcomes. Replay reproduced exact metrics for all 440 runs. The 110 stored trace instances include deliberately reused realizations for paired component ablations; policy outcomes are not treated as independent statistical samples.

Code review identified and corrected configuration-option precedence and coincident-event routing issues. Completion events now release resources before simultaneous arrivals and transfers are routed. The reference suite was rerun after this correction and its numerical output hashes remained unchanged. Validation establishes internal consistency and determinism rather than physical-model accuracy.

9. Conclusion

SmartDistServe provides a reproducible simulation artifact for evaluating phase-local scheduling in disaggregated LLM serving. Within the tested synthetic workload grid, the slack, duration, and aging heuristic improves mean joint SLO attainment over FCFS in four conditions, reduces it in four, and ties in eight. Estimated SJF can achieve substantially higher attainment under overload while increasing tail waiting. These findings support evaluation of multiple performance and fairness measures rather than a claim of universal policy superiority. The principal contribution is the experimental integration and its inspectable measured tradeoffs; algorithmic novelty and improvement over the physical DistServe system are not established. Empirical traces, calibrated service models, stronger baselines, and physical GPU evaluation remain necessary for external validation.

References

[1] Zhong et al. (2024). DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. OSDI 2024.

https://www.usenix.org/system/files/osdi24-zhong-yinmin.pdf

[2] Qiu et al. (2024). Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction. arXiv:2404.08509.

https://arxiv.org/abs/2404.08509

[3] Fu et al. (2024). Efficient LLM Scheduling by Learning to Rank. arXiv:2408.15792.

https://arxiv.org/abs/2408.15792

[4] Sun et al. (2024). Llumnix: Dynamic Scheduling for Large Language Model Serving. OSDI 2024.

https://www.usenix.org/conference/osdi24/presentation/sun-biao

[5] Huang et al. (2025). SLO-Aware Scheduling for Large Language Model Inferences. arXiv:2504.14966.

https://arxiv.org/abs/2504.14966

[6] Wu et al. (2026). FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference. NSDI 2026.

https://www.usenix.org/conference/nsdi26/presentation/wu-bingyang

Appendix A. Reproduction Procedure

The following commands execute from the repository root. Simulation, tests, auditing, and reporting require Python 3.11 or later and the standard library. Plot generation requires the pinned dependencies in requirements-plots.txt. The reference manifest records the precise Python environment and source hashes.

```
python -m unittest discover -s tests -v
python -m smartdistserve experiment --config configs/experiment.json
python -m smartdistserve report --results results/reference
python scripts/audit_results.py --results results/reference
python scripts/replay_results.py --results results/reference
python scripts/plot_results.py --results results/reference
```

The saved manifest contains all workload definitions, seeds, hardware and scheduler settings, source hashes, trace hashes, and output hashes. Complete inputs are retained in traces.jsonl.gz; policy-specific timestamps, measured-cohort flags, and success outcomes are retained in requests.csv.gz. Seed-level metrics, group statistics, aggregate uncertainty, and paired contrasts are retained separately. All requests are drained; failed or late requests are not censored.

Appendix B. Measured Figures

The following six figures are generated from the saved run metrics. Their error bars represent 95% Student t intervals across five seeds, not uncertainty in the hardware cost model. Figures use zero-baseline axes; unconstrained intervals can extend outside natural metric ranges, and complete numeric bounds remain available in summary.csv. Landscape pages preserve figure legibility.

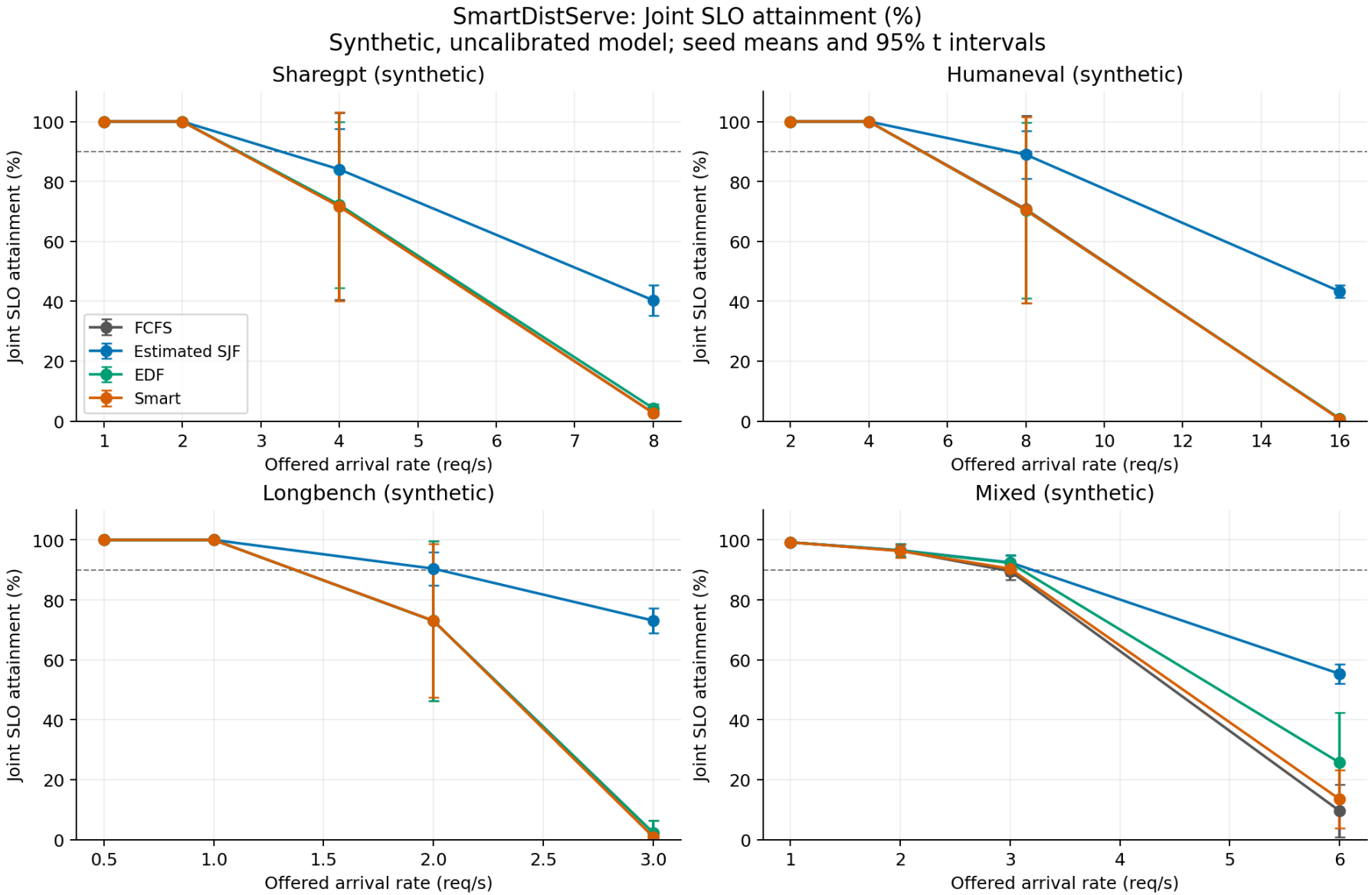


Figure 1. Joint SLO attainment across all main workload profiles and offered rates.

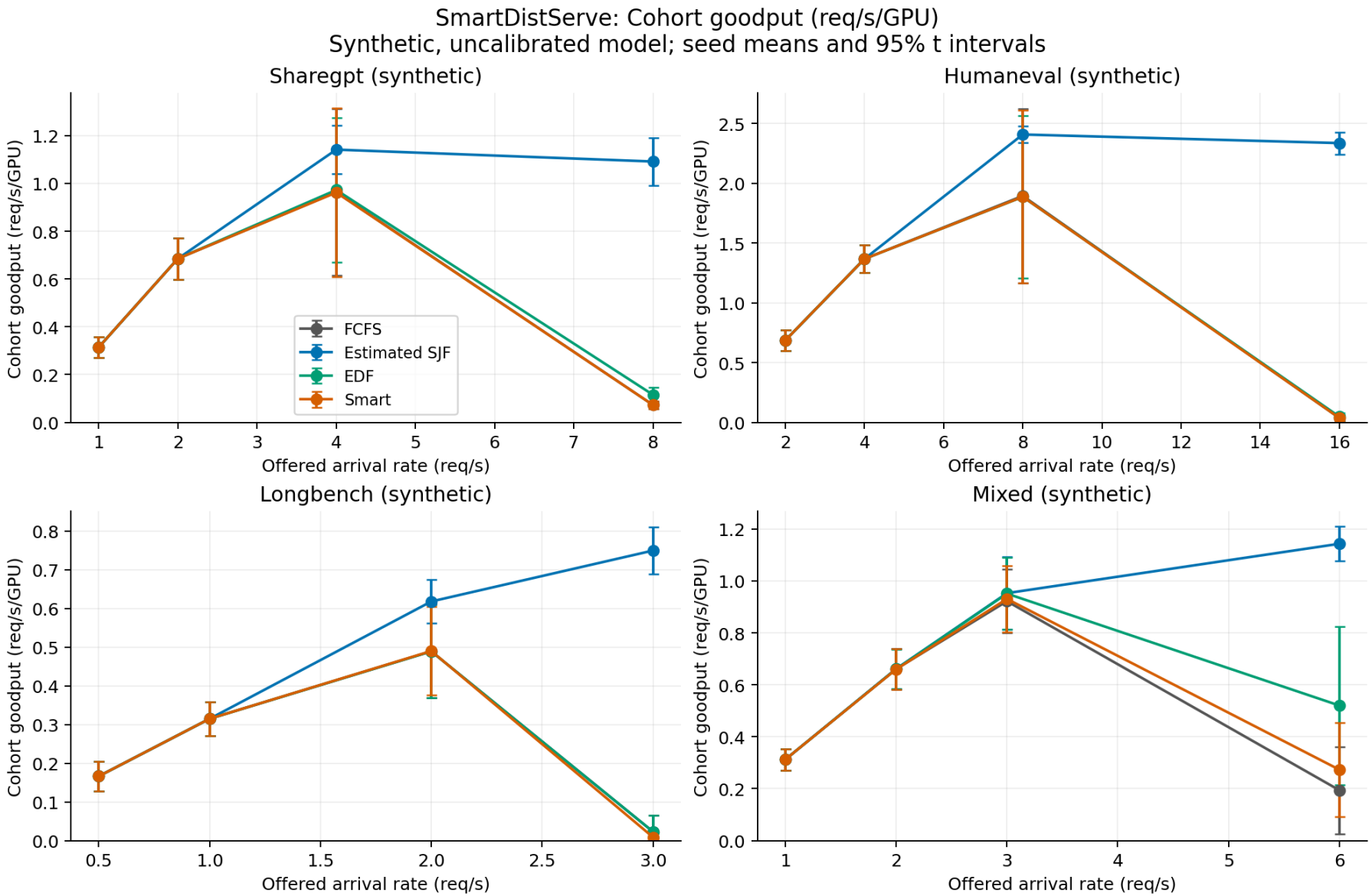


Figure 2. Successful cohort-request rate per provisioned GPU. This is not sustainable steady-state capacity.

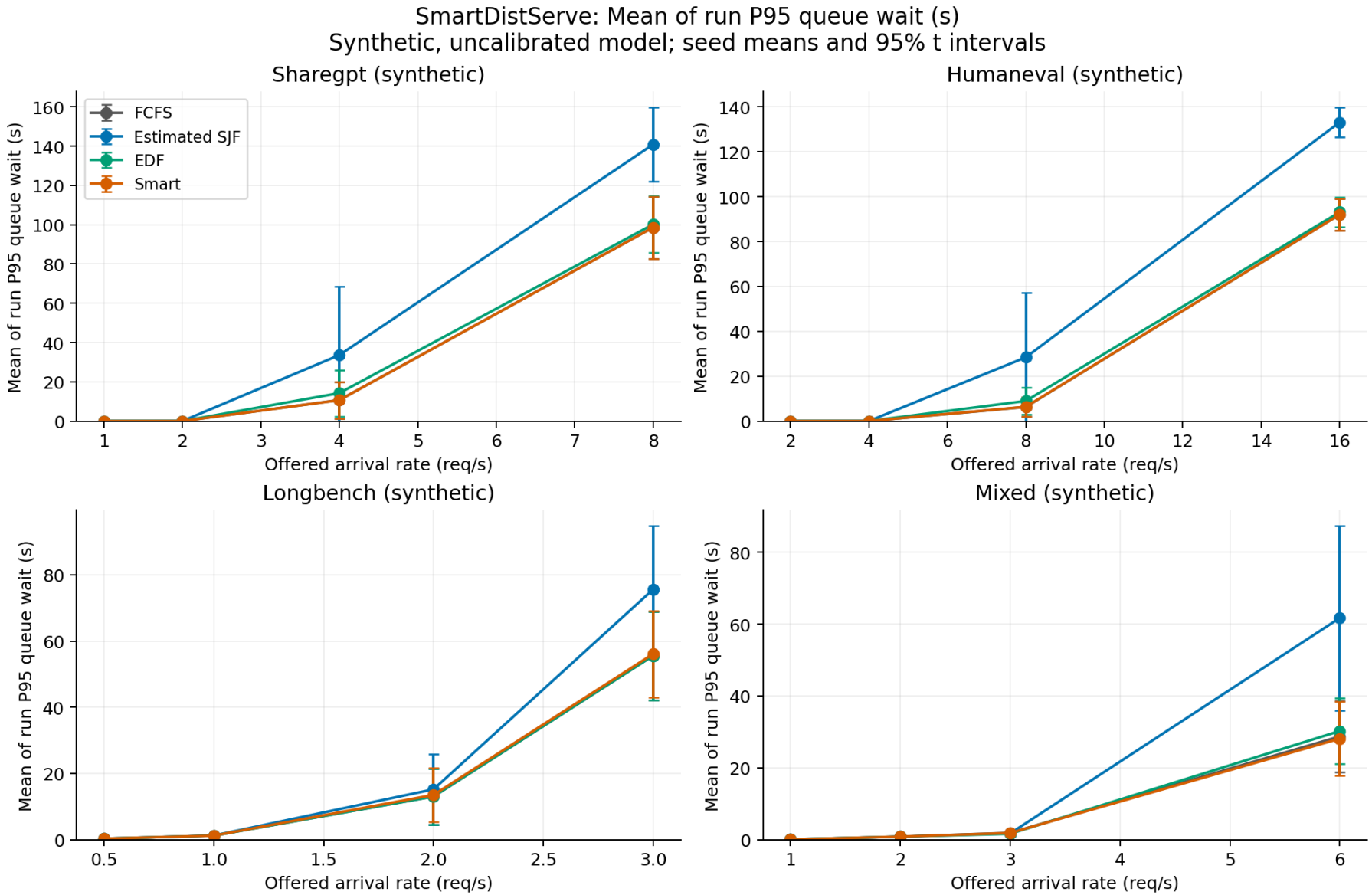


Figure 3. Mean of run-level P95 queue waiting times. Transfer and active service are excluded.

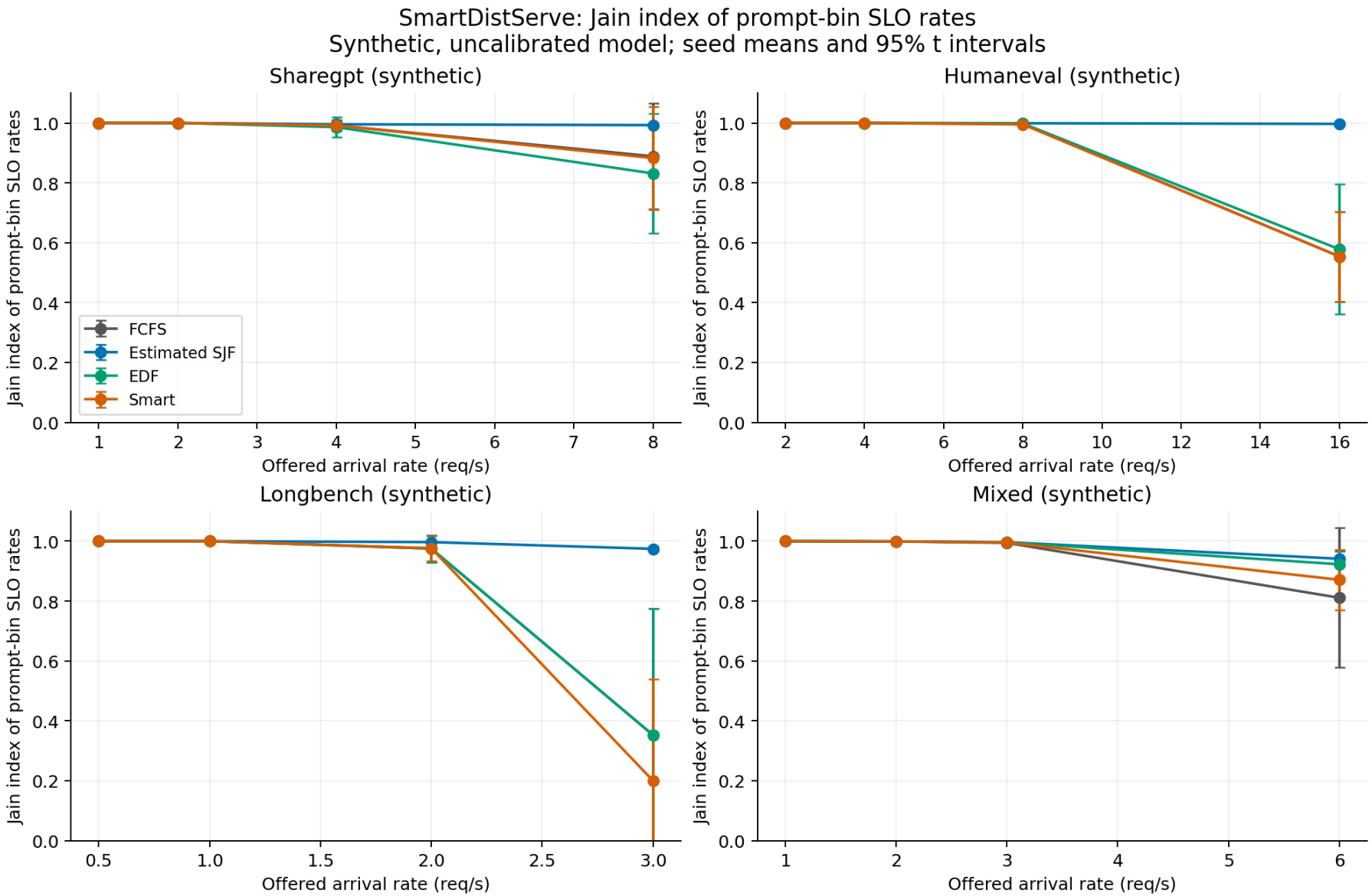


Figure 4. Jain index of prompt-bin SLO attainment rates. Outcome parity does not establish resource fairness.

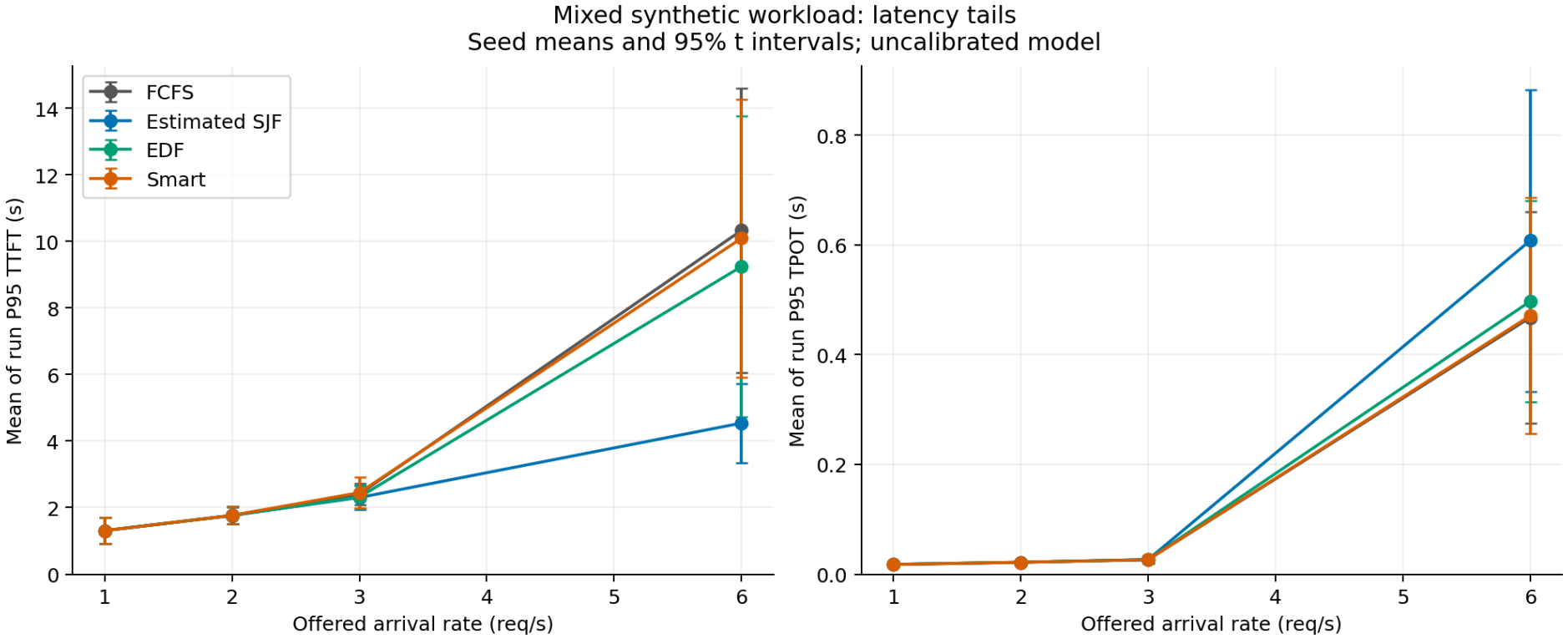


Figure 5. Mean run-level P95 TTFT and TPOT under the mixed workload.

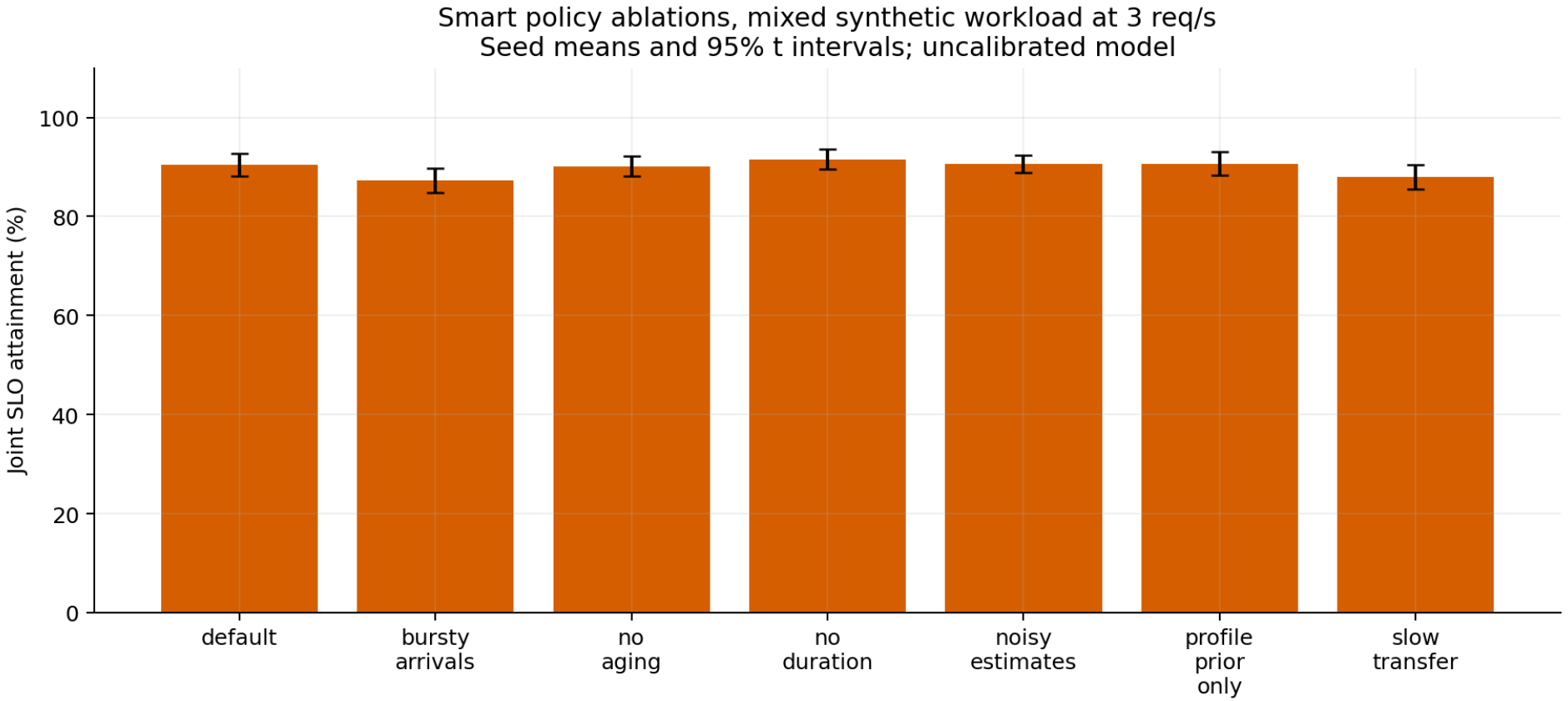


Figure 6. Smart-policy component and sensitivity ablations at mixed rate 3 requests per second.